Project Overview: Library Management API

We will build an API for managing books in a library, demonstrating all routing concepts:

- ⇨ Convention-based routing
- ⇨ Attribute based routing
- ⇨ Route parameters & constraints
- ⇨ HTTP methods (GET , POST, PUT, DELETE)
- ⇨ Query strings
- ⇨ LINQ queries

Step 1: Create the Project

Now we have to open the terminal to the exact folder (or navigate to the particular folder) where we are wanting to create the project. And after creating we will navigate to the API project folder.

Opening the terminal,

Create the project,

Command 1: dotnet new webapi -n LibraryApi -f net8.0

Navigate to api folder,

Command 2: cd LibraryApi

Step 2: Install required packages

Commands:

1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

5: dotnet add package Swashbuckle.AspNetCore --version 6.4.0

Step 3: Create the Model (Entity)

Path: LibraryApi > Models > Book.cs

```
Book.cs
Models > Book.cs > ...
namespace LibraryApi.Models
{
    0 references
    public class Book
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Title { get; set; } = string.Empty;
        0 references
        public string Author { get; set; } = string.Empty;
        0 references
        public string Genre { get; set; } = string.Empty;
        0 references
        public int PublishedYear { get; set; }
        0 references
        public bool IsAvailable { get; set; } = true;
        0 references
        public decimal Price { get; set; }
    }
}
```

Step 4: Create Database Context

Path: LibraryApi>Data>ApplicationDbContext.cs

```
ApplicationDbContext.cs
Data > ApplicationDbContext.cs > ...
// To inherite the DBContext class
using LibraryApi.Models;
using Microsoft.EntityFrameworkCore;

namespace LibraryApi.Data
{
    2 references
    public class ApplicationDbContext : DbContext
    {
        0 references
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
        options) : base(options)
        {

        }

        0 references
        public DbSet<Book> Books{ get; set; }
    }
}
```

Step 5: Configure the Database Connection

By adding the connection string in **appsetting.json**:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=Farabi;Initial Catalog=LibraryDb_1;Integrated Security=True;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Step 6: Configure Services in Program.cs

Path: LibraryApi > Program.cs

Program.cs

```
using LibraryApi.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Add database context
builder.Services.AddDbContext<ApplicationDbContext>
(options => options.UseSqlServer
    (builder.Configuration.GetConnectionString
        ("DefaultConnection")));

var app = builder.Build();

// Configre the http request pipeline
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

// Seed some initial data
using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider
    .GetRequiredService<ApplicationDbContext>();
    dbContext.Database.EnsureCreated();
    // Creates database if not exists
}

app.Run();
```

Step 7: LINQ Basics

What is LINQ?

LINQ (Language Integrated Query) allows us to query collections (like database tables) using C# syntax.

Common LINQ Methods We Will Use:

.ToList()	Converts results to a list	db.Books.ToList()
.Where()	Filters data	.Where(b => b.Price > 20)
.FirstOrDefault()	Gets first item or null	.FirstOrDefault(b => b.Id == 5)
.OrderBy()	Sorts data	.OrderBy(b => b.Title)
.Any()	Checks if any exists	.Any(b => b.IsAvailable)
.Count()	Count items	.Count(b => b.Genre == “Fiction”)

LINQ Example:

```
// Without LINQ (old way)
List<Book> expensiveBooks = new List<Book>();
foreach (var book in allBooks)
{
    if (book.Price > 20)
        expensiveBooks.Add(book);
}

// With LINQ (simple!)
var expensiveBooks = allBooks.Where(b => b.Price > 20).ToList();
```

The b => b.Price > 20 is a **lambda expression** - read as "for each book, check if price > 20".

Step 8: Create Controller with ALL Routing Concepts

Path: LibraryApi > Controllers > BooksController.cs

BooksController.cs

Controllers > BooksController.cs > { } LibraryApi.Controllers > BooksController > GetLibraryStarts() : Tas

```
using LibraryApi.Data;
using LibraryApi.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LibraryApi.Controllers
{
    // ==========================================================
    // Section 1: ATTRIBUTE ROUTING (Preferred for webapi)
    // ==========================================================
    [ApiController]
    [Route("api/[controller]")]
    // Route: api/books
    1 reference
    public class BooksController : ControllerBase
    {
        15 references
        private readonly ApplicationDbContext _context;
        0 references
        public BooksController(ApplicationDbContext context)
        {
            _context = context;
        }

        // ==========================================================
        // GET: api/books
        // Return all books (using LINQ)
        // ==========================================================
        [HttpGet]
        0 references
        public async Task<ActionResult<IEnumerable<Book>>> GetAllBooks()
        {
            // LINQ: Get all books from database
            var books = await _context.Books.ToListAsync();
            return Ok(books);
        }
```

```
        // ========================================================
        // GET: api/books/available
        // Returns only available books (Custom route with LINQ Where)
        [HttpGet("available")]
        0 references
        public async Task<ActionResult<IEnumerable<Book>>> GetAvailableBooks()
        {
            // LINQ: Filter books that are available
            var availableBooks = await _context.Books.Where(b => b.IsAvailable == true)
                                                     .ToListAsync();
            return Ok(availableBooks);
        }

        // ========================================================
        // GET: api/books/genre/{genre}
        // Returns books by genre (Route parameter with LINQ)
        // ========================================================
        [HttpGet("genre/{genre}")]
        0 references
        public async Task<ActionResult<IEnumerable<Book>>> GetBooksByGenre(string genre)
        {
            // LINQ: Filter by genre (case-insensitive)
            var books = await _context.Books.Where(b => b.Genre
                                                        .ToLower() == genre
                                                        .ToLower()).ToListAsync();
            if (!books.Any())
            {
                return NotFound($"No books found in genre: {genre}");
            }

            return Ok(books);
        }
```

```
        // ========================================================
        // GET: api/books/{id}
        // Returns single book by ID (Route parameter)
        // ========================================================
        [HttpGet("{id : int}")] // :int is a route constraint
        1 reference
        public async Task<ActionResult<Book>> GetBookById(int id)
        {
            // LINQ: Find specific book by ID
            var book = await _context.Books.FirstOrDefaultAsync(b => b.Id == id);

            if(book == null)
            {
                return NotFound($"Book with ID {id} not found");
            }

            return Ok(book);
        }
```

```
        // ========================================================
        // GET: api/books/search?author=King&year=2020
        // Demonstrates QUERY STRING parameters
        // ========================================================
        [HttpGet("search")]
        0 references
        public async Task<ActionResult<IEnumerable<Book>>> SearchBooks
        ([FromQuery] string? author, [FromQuery] int? year, [FromQuery] string? genre)
        {
            // LINQ: Start with all books
            var query = _context.Books.AsQueryable();

            // Apply filters only if parameters are provided
            if (!string.IsNullOrEmpty(author))
            {
                query = query.Where(b => b.Author.Contains(author));
            }

            if (year.HasValue)
            {
                query = query.Where(b => b.PublishedYear == year.Value);
            }

            if (!string.IsNullOrEmpty(genre))
            {
                query = query.Where(b => b.Genre.Contains(genre));
            }

            var results = await query.ToListAsync();

            if (!results.Any())
            {
                return NotFound("No books match your search criteria");
            }

            return Ok(results);
        }
```

```
119
120         // ============================================================
121         // POST: api/books
122         // Creates a new book (demonstrates model building from body)
123         // ============================================================
124         [HttpPost]
            0 references
125         public async Task<ActionResult<Book>> CreateBook([FromBody] Book newBook)
126         {
127             // LINQ: Check if book with same title exists
128             bool exists = await _context.Books.
129             AnyAsync(b => b.Title.ToLower() == newBook.Title.ToLower());
130
131             if (exists)
132             {
133                 return BadRequest("A book with this title already exists");
134             }
135
136             // Add new book
137             _context.Books.Add(newBook);
138             await _context.SaveChangesAsync();
139
140             // Returns 201 Created with Location header
141             return CreatedAtAction(nameof(GetBookById), new { id = newBook.Id },
142             newBook);
143         }
144
```

```
144
145         // ============================================================
146         // PUT: api/books/{id}
147         // Updates an entire book
148         // ============================================================
149         [HttpPut("{id}")]
            0 references
150         public async Task<ActionResult> UpdateBook(int id, [FromBody] Book updatedBook)
151         {
152             if (id != updatedBook.Id)
153             {
154                 return BadRequest("ID mismatch");
155             }
156
157             // LINQ: Check if book exists
158             var existingBook = await _context.Books.FirstOrDefaultAsync(b => b.Id == id);
159
160             if (existingBook == null)
161             {
162                 return NotFound($"Book with ID {id} not found");
163             }
164
165             // Update properties
166             existingBook.Title = updatedBook.Title;
167             existingBook.Author = updatedBook.Author;
168             existingBook.Genre = updatedBook.Genre;
169             existingBook.PublishedYear = updatedBook.PublishedYear;
170             existingBook.IsAvailable = updatedBook.IsAvailable;
171             existingBook.Price = updatedBook.Price;
172
173             await _context.SaveChangesAsync();
174
175             return Ok(new { message = "Book updated successfully", book = existingBook});
176         }
177
```

```
177
178         // ============================================================
179         // DELETE: api/books/{id}
180         // Delete a book
181         // ============================================================
182         [HttpDelete("{id}")]
            0 references
183         public async Task<IActionResult> DeleteBook(int id)
184         {
185             // LINQ: Find book
186             var book = await _context.Books.FirstOrDefaultAsync(b => b.Id == id);
187
188             if (book == null)
189             {
190                 return NotFound($"Book with ID {id} not found");
191             }
192
193             _context.Books.Remove(book);
194             await _context.SaveChangesAsync();
195
196             return Ok(new { message = $"Book '{book.Title}' deleted successfully"});
197         }
198
```

```
        // ==============================================================
        // GET: api/books/starts
        // Shows various LINQ aggregate functions
        // ==============================================================
        [HttpGet("starts")]
        0 references
        public async Task<IActionResult> GetLibraryStarts()
        {
            var allBooks = await _context.Books.ToListAsync();

            // Demonstrate various LINQ methods
            var starts = new
            {
                TotalBooks = allBooks.Count,
                AvailableBooks = allBooks.Count(b => b.IsAvailable),
                CheckedOutBooks = allBooks.Count(b => !b.IsAvailable),
                AveragePrice = allBooks.Average(b => b.Price),
                MostExpensiveBook = allBooks.Max(b => b.Price),
                CheapestBook = allBooks.Min(b => b.Price),
                UniqueGenres = allBooks.Select(b => b.Genre).Distinct().Count(),
                BooksByYear = allBooks.GroupBy(b => b.PublishedYear).Select(g =>
                    new { Year = g.Key, Count = g.Count() }).OrderByDescending(g => g.Year)
            };

            return Ok(starts);
        }
    }
}
```

Step 9: Demonstrate Convention-Based Routing

Now let's create a second controller that uses **Convention-Based Routing** (just for learning comparison).

Path: LibraryApi > Controllers > ReportsController.cs

ReportsController.cs

Controllers > ReportsController.cs > ...

```
using LibraryApi.Data;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LibraryApi.Controllers
{
    // NO [ApiController] or [Route] attributes here!
    // This controller will use convention-based routing from Program.cs
    1 reference
    public class ReportsController : ControllerBase
    {
        4 references
        private readonly ApplicationDbContext _context;

        0 references
        public ReportsController(ApplicationDbContext context)
        {
            _context = context;
        }

        // Will be accessible via: api/reports/summary
        [HttpGet]
        0 references
        public async Task<IActionResult> Summary()
        {
            var totalBooks = await _context.Books.CountAsync();
            var availableBooks = await _context.Books.CountAsync(b => b.IsAvailable);

            return Ok(new
            {
                TotalBooks = totalBooks,
                AvailableBooks = availableBooks,
                AvailablityRate = totalBooks > 0 ? (availableBooks * 100 / totalBooks) : 0
            });
        }

        // Will be accessible via: api/reports/byauthor/jkrowling
        [HttpGet("byauthor/{author}")]
        0 references
        public async Task<IActionResult> GetBooksByAuthor(string author)
        {
            var books = await _context.Books.Where(b => b.Author.Contains(author)).ToListAsync();

            return Ok(books);
        }
    }
}
```

Now update Program.cs to add convention-based routing **alongside** attribute routing:

// Add this AFTER app.MapControllers(); but BEFORE app.Run();

// This adds convention-based routing as an alternative

app.MapControllerRoute(

name: "default",

pattern: "api/{controller}/{action}/{id?}");

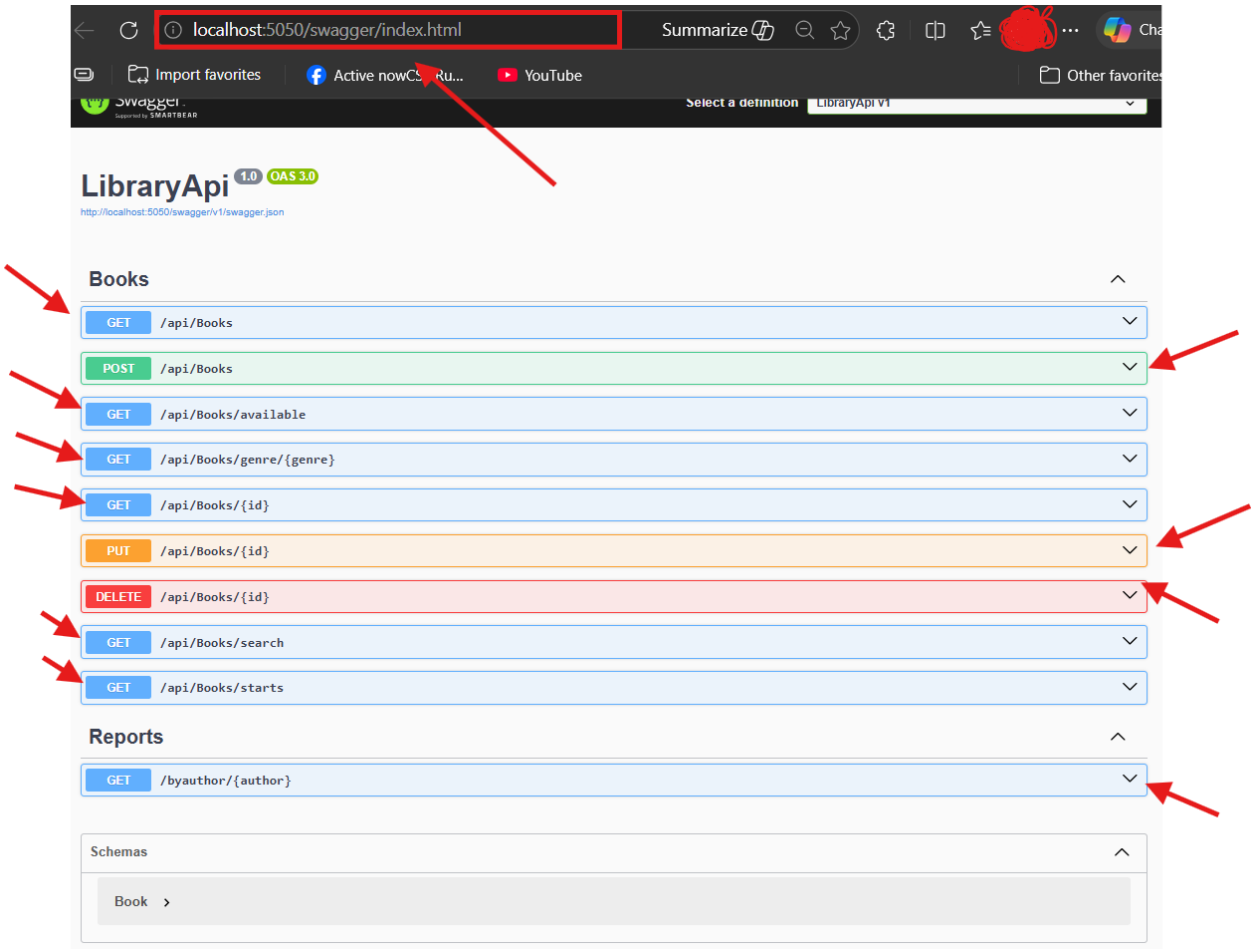
Now Program.cs:

Program.cs

```
using LibraryApi.Data;
using LibraryApi.Models;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Add database context
builder.Services.AddDbContext<ApplicationDbContext>
(options => options.UseSqlServer
    (builder.Configuration.GetConnectionString
        ("DefaultConnection")));

var app = builder.Build();

// Configre the http request pipeline
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

```
26
27  app.UseHttpsRedirection();
28  app.UseAuthorization();
29  app.MapControllers();   // For atribute routing
30
31  // Aff this for convention-based routing
32  app.MapControllerRoute(
33      name: "default",
34      pattern: "api/{controller}/[action]/{id?}"
35  );
36
37  // Seed some initial data
38  using (var scope = app.Services.CreateScope())
39  {
40      var dbContext = scope.ServiceProvider
41      .GetRequiredService<ApplicationDbContext>();
42      dbContext.Database.EnsureCreated();
43      // Creates database if not exists
44
45      // Add some data if none exists
46      if (!dbContext.Books.Any())
47      {
48          dbContext.Books.AddRange(
49              new Book { Title = "The Hobbit",
50                         Author = "J.R.R. Tolkien",
51                         Genre = "Fantasy",
52                         PublishedYear = 1937,
53                         Price = 15.99m,
54                         IsAvailable = true
55                       },
56              new Book { Title = "1984",
57                         Author = "George Orwell",
58                         Genre = "Dystopian",
59                         PublishedYear = 1949,
60                         Price = 12.99m,
61                         IsAvailable = false
62                       },
63              new Book { Title = "Pride and Prejudice",
64                         Author = "Jane Austen",
65                         Genre = "Romance",
66                         PublishedYear = 1813,
67                         Price = 9.99m,
68                         IsAvailable = true
69                       }
70          );
71
72          dbContext.SaveChanges();
73      }
74  }
75
76  app.Run();
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   ...   dotnet

PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi> dotnet watch run
      SELECT 1 ELSE SELECT 0
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (6ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT CASE
          WHEN EXISTS (
              SELECT 1
              FROM [Books] AS [b]) THEN CAST(1 AS bit)
          ELSE CAST(0 AS bit)
      END
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5050
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\Routing\LibraryApi
```



A result through attribute-based routing,

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5050/api/Books/1' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5050/api/Books/1
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "title": "The Hobbit",
  "author": "J.R.R. Tolkien",
  "genre": "Fantasy",
  "publishedYear": 1937,
  "isAvailable": true,
  "price": 15.99
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat,11 Apr 2026 19:17:02 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "title": "string",
  "author": "string",
  "genre": "string",
  "publishedYear": 0,
  "isAvailable": true,
  "price": 0
}
```

A result through convention-based routing,

GET /byauthor/{author}

Parameters

Cancel

Name	Description
author * required string (path)	J.R.R. Tolkien

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5050/byauthor/J.R.R.%20Tolkien' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5050/byauthor/J.R.R.%20Tolkien
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "title": "The Hobbit",
    "author": "J.R.R. Tolkien",
    "genre": "Fantasy",
    "publishedYear": 1937,
    "isAvailable": true,
    "price": 15.99
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sat,11 Apr 2026 19:18:44 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links